Prepared from the current local repository checkout at `/home/chris/cardvault`.

CardVault Holistic Logic Audit

0. Executive Summary

My high-level assessment:

AREA	ESTIMATED SHARE	STATUS
Sound foundation, worth keeping	65%	GREEN
Usable but approximate or under-modeled	25%	YELLOW
Materially flawed and should be fixed soon	10%	RED

Plain-English summary:

- this project does not need a restart
- it has a solid enough foundation to rehabilitate
- the main weaknesses are structural and semantic, not total architectural failure

What is solid:

- the project shape
- the deadline model
- the general card/store structure
- the alert engine
- the basic formulas themselves

What is not solid:

- source-of-truth consistency
- sync integrity
- confidence labeling for estimated numbers
- some recommendation semantics

My recommendation:

- do not start over
- fix the logic model in place
- preserve the current app structure
- refactor the trust boundaries

0.1 Should the developer restart or repair?

Answer:

- repair, do not restart

Why:

- the app already contains domain structure, useful UI flows, persistence wiring, and business concepts that would be expensive to recreate
- most issues are from inconsistency and approximation, not from an irredeemable architecture

Restart only makes sense if:

- the product scope changes dramatically
- the team wants to replace the entire household/rewards model
- the current code proves impossible to test or reason about after targeted cleanup

That is not the case from what I reviewed.

0.2 What would it take for an LLM to get this back on track?

Answer:

- very feasible, if the work is staged correctly

An LLM could likely get this back on track in 3 phases:

PHASE	GOAL	LLM SUITABILITY
1	unify logic and source-of-truth rules	high
2	repair sync correctness and add scenario tests	high
3	improve recommendation quality and semantic labeling	high

Likely LLM-enabled repair plan:

1. Inventory all displayed metrics and map them to current source data.

2. Introduce a canonical balance-resolution layer.

3. Refactor dashboard totals to use canonical logic.

4. Repair cloud-sync empty-state and per-scope initialization bugs.

5. Add scenario tests around household, loyalty balances, and deletes.

6. Add confidence labels to derived outputs.

7. Reword misleading labels like "portfolio value" and "lifetime signup bonus value."

8. Improve recommendation logic only after trust in source data is restored.

Expected effort for an LLM with good repo access:

- audit + plan: low
- implementation of the highest-value fixes: moderate
- test hardening and UI explanation improvements: moderate

Net:

- this is a strong candidate for iterative LLM-assisted repair
- it is not a good candidate for wholesale re-generation from scratch

0.3 Overall project health judgment

Current health:

- foundation: solid
- trustworthiness of displayed numbers: mixed
- maintainability: fair, but needs clearer source-of-truth rules
- recommendation quality: acceptable for heuristics, not yet strong enough for high-confidence advice

Bottom line:

- the project is closer to "promising but inconsistent" than "broken"

This document is the opinionated version of the logic report.

Purpose:

- explain the logic in business terms
- identify what is mathematically sound
- identify what is heuristic or internally inconsistent
- recommend improvements and explain why they matter

Color key:

- GREEN = logic is coherent and appropriate for the current product
- YELLOW = logic is understandable but approximate, brittle, or incomplete
- RED = logic is likely to mislead users or create incorrect outputs

---

1. Executive View

CardVault is not a heavy financial-modeling system.

At a high level, the product is:

- a household card inventory
- a deadline engine

- a valuation engine based on static CPP assumptions
- a recommendation engine driven by thresholds
- a cloud sync layer that decides what data the formulas see

The biggest project-level truth:

- most "math issues" here are not advanced math bugs
- they are usually one of:
- stale or inconsistent source data
- mixed source-of-truth rules
- heuristic assumptions presented like exact calculations
- business thresholds that are reasonable but not personalized

Overall status:

AREA	STATUS	WHY
Date math	GREEN	Simple, coherent, and mostly consistent
Alert generation	GREEN	Clear thresholds and understandable behavior
Dashboard totals	YELLOW	Straightforward, but not always aligned with richer reward-balance logic
Recommendation engine	YELLOW	Understandable, but threshold-driven and only lightly personalized
Transfer optimization	YELLOW	Internally coherent, but based on static CPP instead of real-world redemption data
Loyalty / household balance model	RED	Different screens use different balance sources, which can create conflicting totals
Cloud sync impact on calculations	RED	Sync edge cases can make valid formulas operate on bad data

---

2. High-Level Logic Model

The project can be understood as a pipeline:

```
stored data -> normalized dates/balances -> totals and deadlines -> recommendations -> UI
```

That means there are four places where the system can go wrong:

STAGE	STATUS	FAILURE MODE
Stored data	RED	stale, duplicated, missing, or wrongly scoped data
Normalization	YELLOW	inferred dates and mixed balance rules
Calculation	GREEN / YELLOW	formulas are usually simple, but assumptions can be rough
Presentation	YELLOW	heuristic values can look more authoritative than they are

---

3. What Makes Sense

3.1 Date arithmetic is mostly solid

Status: GREEN

What it does well:

- uses calendar-day difference instead of naive milliseconds
- uses explicit handling for missing dates
- consistently treats overdue items as negative values

Formula:

- `days_remaining = differenceInCalendarDays(target_date, today_at_local_midnight)`

Why this makes sense:

- annual fee deadlines and signup bonus deadlines are calendar concepts
- exact times of day do not matter here

Improvement:

- replace the `9999` sentinel with `null` or a typed enum in the core domain model

Why:

- `9999` works, but it leaks presentation concerns into logic
- future developers can easily misread it as a real number instead of "no deadline"

3.2 Alert logic is coherent

Status: GREEN

What it does well:

- alert generation is easy to understand

- thresholds are consistent
- rules are not deeply coupled to UI state

Severity formula:

- if `days < 60` -> red urgency
- else if `days <= 120` -> amber urgency
- else -> green urgency

Why this makes sense:

- the app's main value is deadline visibility
- simple time-window thresholds are appropriate for that

Improvement:

- add explicit severity names in the business language:
- `critical`
- `approaching`
- `future`

Why:

- business semantics are easier to discuss than raw color names

3.3 Fee forecasting is reasonable

Status: **GREEN**

Formula:

- `fees_next_12_months = sum(annualFee for active cards where 0 <= days_to_fee <= 365)`

Why this makes sense:

- it is a practical cash-planning metric
- it excludes closed cards and overdue dates

Improvement:

- add two more variants:
- next 90 days
- next 24 months

Why:

- users think about immediate burn and medium-term exposure differently

4. What Is Understandable But Approximate

4.1 Portfolio value math

Status: YELLOW

Formulas:

- `min_value = sum(balance * conservative_cpp) / 100`
- `max_value = sum(balance * optimistic_cpp) / 100`

What makes sense:

- the formula is internally coherent
- cents-per-point math is standard in rewards communities
- min/max framing is better than a single fake-precise number

What does not fully make sense:

- static CPP assumptions are presented as if they are household portfolio value
- actual realizable value depends on:
- transfer partner access
- award availability
- taxes and fees
- route constraints
- traveler-specific behavior

Improvement:

- relabel these as:
- `estimated redemption value floor`
- `estimated redemption value ceiling`

Why:

- "portfolio value" sounds more objective than the data supports

Improvement:

- track `valuation_confidence` per program

Example:

- GREEN for cashback
- YELLOW for flexible currencies
- RED for aspirational airline sweet-spot assumptions

Why:

- not all CPP estimates deserve the same level of trust

4.2 Signup bonus timing estimates

Status: YELLOW

Formula:

- `estimated_sub_received = (approvalDate ?? dateOpened) + subDeadlineMonths + 30 days`

What makes sense:

- it gives the system a conservative planning anchor when actual data is missing

What does not fully make sense:

- statement close timing is not always +30 days
- bonus posting behavior varies widely by issuer and card

Improvement:

- mark this output explicitly as estimated in the UI and reports

Why:

- this is planning math, not factual history

Improvement:

- let users confirm or override estimated SUB received dates

Why:

- replacing inferred dates with actual dates improves every downstream rule

4.3 Recommendation thresholds

Status: YELLOW

Examples:

- fee > 400 triggers review recommendation
- 4+ cards in last 24 months triggers Chase warning
- 100,000 combined points drives one joint-strategy branch

What makes sense:

- threshold logic is fast, explainable, and easy to debug

What does not fully make sense:

- thresholds are not personalized
- thresholds are not derived from user goals, travel patterns, or issuer-specific nuance

Improvement:

- separate recommendations into:
- policy rules
- heuristics
- personalized optimization

Why:

- users can understand which advice is hard constraint vs soft suggestion

---

5. What Does Not Make Sense and Should Be Improved

5.1 Mixed balance source-of-truth rules

Status: RED

Current problem:

- some calculations use `RewardProgram.balance`
- some calculations derive balances from `loyaltyMemberships`
- some screens effectively mix both worlds

This means:

- one screen can say the household has one number
- another screen can imply a different number for the same program family

Why this is a real issue:

- this is not merely a UI quirk
- it changes totals, valuations, transfer recommendations, and trust in the product

What should improve:

- create one canonical balance-resolution layer used by all read paths

Suggested model:

- `getResolvedProgramBalance(programId, scope, personFilter)`
- every screen reads through that function

Why:

- totals, dashboards, and strategy views should all agree on the same balance logic

Suggested add-on:

- add a `balance_source` field to displayed metrics

Examples:

- `household currency`
- `derived from loyalty memberships`
- `manual estimate`

Why:

- makes the origin of the number visible instead of hidden

5.2 Cloud sync can undermine correct math

Status: **RED**

Current problem:

- the sync layer can leave stale local data around
- the initial sync state is device-level rather than user/scope-level
- some delete flows do not reliably clear data everywhere

Why this matters:

- if the card set is wrong, every formula downstream becomes wrong

This is the hierarchy of truth:

- data integrity is more important than formula correctness

Improvement:

- make sync state keyed by user + household scope

Formula idea:

- `sync_key = user_id + scope_mode + scope_id`

Why:

- initial sync should not be shared across unrelated users or household states on the same browser

Improvement:

- treat empty cloud result as a valid state, not a no-op

Why:

- deleting the last record should actually clear the local store

Improvement:

- add a visible sync status panel in Settings

Include:

- current scope
- last cloud fetch time
- pending queue count
- initial sync complete flag

Why:

- makes "wrong number" debugging much faster

5.3 Dashboard totals are too disconnected from the richer rewards model

Status: RED

Current problem:

- dashboard totals are simple sums over `rewardPrograms`
- transfer logic uses more nuanced loyalty-membership derivation

Why this is bad:

- the dashboard is the most trusted summary view
- if it is simpler than the deeper logic, it can be the most misleading screen in the app

Improvement:

- rebuild dashboard totals on top of the same resolved balance layer used by transfer logic

Why:

- summary views should be downstream of canonical logic, not parallel to it

---

6. Formula-by-Formula Audit

6.1 Active card totals

Formula:

- `active_cards = cards where status = active`
- `total_cards = count(active_cards)`

Status: GREEN

Assessment:

- simple and correct for the product

Improvement:

- optionally expose `total_open_cards` and `total_all_cards`

Why:

- some users may want active-only vs all-history counts separated

6.2 Cards needing action

Formula:

- include if `status != closed` and (`days_to_fee <= 60` or `keepCancelStatus = cancel`)

Status: GREEN

Assessment:

- good attention metric

Improvement:

- split into two displayed subcounts:
- fee action
- cancel action

Why:

- one blended number hides the actual cause of urgency

6.3 Upcoming fee list

Formula:

- include if `annualFee > 0` and `days_to_fee <= 120`

Status: YELLOW

Assessment:

- makes sense for the list view
- less ideal as a universal "upcoming" definition

Improvement:

- parameterize the horizon instead of hardcoding 120

Why:

- different screens may need 60, 90, 120, or 365 days

6.4 Lifetime signup bonus value

Formula:

- `sum(enrollBonusValue ?? 0 for active cards)`

Status: YELLOW

Assessment:

- this is easy to compute, but semantically fuzzy

Why it is weak:

- "lifetime" is a misleading label if only active cards are included

Improvement:

- rename one of these:
- `active_cards_signup_bonus_value`
- `historical_signup_bonus_value`

Why:

- current naming can confuse scope and time horizon

6.5 High-fee recommendation

Rule:

- if `annualFee > 400` and user currently marks keep -> recommend review

Status: YELLOW

Assessment:

- reasonable heuristic
- not enough for strong advice

Improvement:

- compute a rough benefit-retention score

Example:

- `net_card_value = estimated_used_benefits + retention_value + sub_remaining_value - annual_fee`

Why:

- high fee alone is not a meaningful proxy for poor value

6.6 Chase 5/24 rule

Formula:

- count cards opened in last 24 months excluding AU cards

Status: YELLOW

Assessment:

- understandable and close to common hobby shorthand

Weakness:

- uses `dateOpened`, not a true application or account-open policy date

Improvement:

- prefer resolved application date when available

Formula:

- `application_anchor = resolveApplicationDate(card) ?? dateOpened`

Why:

- better matches the rest of the project's attempt to distinguish confidence levels

6.7 Transfer optimization

Formula:

- `effective_cpp = destination_cpp * (toMultiplier / fromMultiplier)`
- recommend transfer if `effective_cpp > cash_cpp`

Status: **YELLOW**

Assessment:

- mathematically coherent for first-pass optimization

Weakness:

- not enough to call something the "best" real-world transfer

Improvement:

- rename result to `best static value route`

Why:

- "best transfer" implies itinerary-aware optimization, which this is not

Improvement:

- factor in transfer fee penalty when present

Possible formula:

- `net_transfer_value = transfer_value - estimated_transfer_fee`

Why:

- currently fee awareness is mostly descriptive, not part of route ranking

---

7. Logic That Should Be Reframed, Not Just Fixed

7.1 The product should separate exact facts from planning assumptions

Status: RED

Right now the project often mixes:

- factual data
- inferred dates
- heuristic guidance
- static valuation assumptions

Improvement:

- add a confidence taxonomy to every derived metric

Suggested categories:

- `exact`
- `derived`
- `heuristic`
- `policy_rule`

Why:

- users can understand whether a number is measured, inferred, or advisory

7.2 The project needs a visible "logic hierarchy"

Status: YELLOW

Recommended hierarchy:

1. Source data integrity

2. Scope and sync integrity

3. Balance resolution

4. Core derived metrics

5. Recommendations and heuristics

Why:

- this is the order in which trust should be established

8. Suggested Additions To Improve Understanding

Yes. There are several additions that would materially improve understanding of both logic and formulas.

8.1 Add a source-of-truth matrix

Status: GREEN

What to add:

METRIC / SCREEN	DATA SOURCE	FORMULA SOURCE	CONFIDENCE	OWNER

Why:

- this would make it obvious where each number comes from

8.2 Add worked examples

Status: GREEN

What to add:

- one card example
- one annual-fee example
- one signup bonus example
- one transfer example
- one household-vs-person balance example

Why:

- formulas become much easier to reason about with concrete numbers

8.3 Add a "fact vs estimate" legend to the UI and reports

Status: GREEN

Suggested display:

- Fact
- Derived
- Estimate
- Heuristic

Why:

- prevents users from over-trusting approximate outputs

8.4 Add scenario tests

Status: YELLOW

What to add:

- whole-household fixture tests instead of only unit tests on helpers

Examples:

- delete last reward program
- move from solo to household
- loyalty membership totals vs dashboard totals
- fee forecast for mixed active/closed cards

Why:

- many correctness issues here are cross-layer, not single-function bugs

8.5 Add a metric glossary

Status: GREEN

Examples:

- `total points`
- `portfolio value`
- `lifetime signup bonus value`
- `needs action`
- `best transfer`

Why:

- several current labels sound more precise than their underlying logic

8.6 Add a "why this recommendation exists" drilldown

Status: GREEN

For each recommendation, show:

- inputs used
- formula or rule
- assumptions
- confidence

Why:

- recommendation trust rises sharply when users can inspect the basis

9. Best Next Improvements

If the goal is maximum correctness per engineering hour, do these in order:

PRIORITY	IMPROVEMENT	STATUS	WHY
1	unify reward-balance resolution behind one canonical function	RED	fixes conflicting totals and downstream logic drift
2	fix cloud-sync empty-state and per-scope initialization behavior	RED	bad data invalidates every formula
3	relabel valuation outputs as estimates with confidence	YELLOW	improves honesty and reduces user confusion
4	move dashboard totals onto canonical balance resolution	RED	summary screen should match deeper logic
5	add scenario-level tests for household, sync, and loyalty cases	YELLOW	catches real-world regressions better than helper tests
6	replace blunt fee-only recommendation with net-value model	YELLOW	materially improves usefulness of advice
7	add source-of-truth matrix and worked examples to docs	GREEN	improves maintainability and auditability

10. Top 10 Logic Fixes

These are ordered by impact on trust and correctness.

RANK	FIX	STATUS	ESTIMATED EFFORT	WHY IT MATTERS
1	unify reward-balance resolution into one canonical read path	RED	1-2 days	removes conflicting totals across screens
2	fix cloud-sync empty-state handling	RED	0.5-1 day	prevents deleted data from lingering locally
3	make sync initialization keyed by user and scope, not just device	RED	0.5-1.5 days	stops cross-user and cross-household contamination
4	move dashboard totals onto canonical balance logic	RED	0.5-1 day	summary view must match deeper logic
5	relabel estimated value outputs with confidence levels	YELLOW	0.5 day	reduces false precision
6	replace "lifetime signup bonus value" with clearer scoped metrics	YELLOW	0.25-0.5 day	improves semantic correctness
7	add scenario tests for household moves, deletes, and loyalty balance reads	YELLOW	1-2 days	catches real-world failures that helper tests miss
8	incorporate transfer fees into route ranking, not just description	YELLOW	0.5-1 day	improves recommendation realism
9	switch Chase 5/24 logic to prefer resolved application dates	YELLOW	0.5 day	better reflects actual eligibility intent
10	split recommendation types into policy rule vs heuristic vs estimate	GREEN	0.5-1 day	makes advice easier to trust and explain

10.1 Suggested implementation sequence

Recommended order:

1. Fix sync correctness.

2. Fix balance source-of-truth.

3. Rebuild dashboard totals.

4. Add scenario tests.

5. Improve semantics and confidence labeling.

6. Improve recommendation sophistication.

Why:

- there is no value in making recommendations smarter if the underlying numbers are still inconsistent

11. Numbers Most Likely Wrong Today

These are the outputs I would distrust first without further validation.

NUMBER / METRIC	STATUS	WHY IT IS AT RISK
dashboard total points	RED	may disagree with loyalty-membership-derived balances
dashboard portfolio value	RED	depends on possibly inconsistent balances plus static CPP assumptions
top reward programs by value	RED	uses simple `rewardPrograms.balance` path and optimistic CPP only
any metric right after sync scope changes	RED	household/individual transitions can leave data in a bad state
any metric after deleting the last card or last reward program on another device	RED	empty cloud results may not clear local state
lifetime signup bonus value	YELLOW	label is misleading relative to the actual inclusion rules
Chase 5/24 count	YELLOW	uses `dateOpened` instead of best-available application-date logic
transfer recommendation "best route"	YELLOW	static-value best route is not the same as best real-world redemption
first-year fee waiver expiry timing	YELLOW	fallback date math is reasonable but still inferred
cards needing action	GREEN	simple and likely correct if source card data is correct

11.1 Numbers I would trust most today

NUMBER / METRIC	STATUS	WHY
annual fee alert countdowns	GREEN	simple date arithmetic
fee due in next 12 months	GREEN	clear inclusion criteria
cancel-marked card counts	GREEN	direct state read
active card counts	GREEN	direct filtered count

12. Bottom Line

The project has good bones:

- the core formulas are usually simple and intelligible
- the alert engine is coherent
- the deadline math is solid

The real weaknesses are structural:

- inconsistent data-source rules
- heuristic outputs that are easy to over-trust
- sync behavior that can poison otherwise-correct calculations

The best way to improve trust is not to make the math more complex first.

It is to:

- unify source-of-truth rules
- separate facts from estimates
- make assumptions visible
- test whole scenarios, not just helper functions